Département EEA – Faculté Sciences et Ingénierie



M1 EEA – Parcours Systèmes et Microsystèmes Embarqués

Rapport de Bureau d'Études VHDL / SOPC

Architecture SOPC & Fonction Simple d'Acquisition de Cap

Année universitaire 2026–2027

Rédigé par **ARIEF Audy** et **NSHUTI NKURIKIYINKA Fidèle**

Binôme : **B6**

19 octobre 2026

Table des matières

1 Introduction et Cahier des Charges **2**
1.1 Principe de la boussole électronique CMPS03 / CMPS10 2
1.2 Fonctionnalités requises du module 2

2 Conception VHDL du Module Compas (`compass_choix_carte`) **2**
2.1 Génération de la base de temps et de l'échantillonnage à 100 kHz 2
2.2 Machine à états finis (FSM) 2
2.3 Décodage de la largeur d'impulsion 3

3 Simulation Temporelle sous ModelSim **3**
3.1 Analyse des résultats de simulation 4

4 Intégration SOPC & Interfaces Avalon-MM **4**
4.1 Composant esclave Avalon : `avalon_aquisition_cap` 4
4.2 Module d'émulation : `avalon_pwm` 4
4.3 Architecture globale sous Qsys / Platform Designer 5

5 Schéma Bloc Quartus et Affectation des Broches **5**
5.1 Schéma Bloc Top-Level (BDF) 5
5.2 Affectation des broches (Pin Planner) 6

6 Développement Logiciel Nios II et Validation **7**
6.1 Programme applicatif C (`hello_world_small.c`) 7
6.2 Résultats d'exécution sur la Console Nios II 8
6.3 Interprétation physique de la mesure 8

7 Conclusion **9**

1 Introduction et Cahier des Charges

Dans le cadre du bureau d'études dédié à la conception d'un pilote automatique de barre franche, ce travail porte sur la conception, la modélisation en VHDL, la simulation temporelle et l'intégration sur cible SOPC du module d'acquisition du cap réel du navire.

1.1 Principe de la boussole électronique CMPS03 / CMPS10

La centrale de cap repose sur un module magnétique (CMPS03 ou CMPS10) fournissant une information d'orientation par modulation de largeur d'impulsion (PWM) :

— La période nominale du signal est typiquement comprise entre 60 ms et 100 ms.
— La durée de l'état haut (t_{haut}) varie linéairement de 1,0 ms (pour un angle de 0°) jusqu'à 36,9 ms (pour un angle de 359°).
— La résolution temporelle est donc de 100 μs par degré supplémentaire :

$$t_{\text{haut}} = 1{,}0 \text{ ms} + (\text{Angle} \times 100\,\mu\text{s}) \tag{1}$$

— Le résultat est numérisé et délivré sous la forme d'un vecteur binaire sur 9 bits (`data_compas[8..0]`), couvrant l'intervalle entier [0 ; 359] degrés.

1.2 Fonctionnalités requises du module

Le module VHDL doit gérer deux modes d'acquisition :

1. **Mode monocoup** (`continu = '0'`) : Une mesure est déclenchée par une impulsion sur l'entrée `start_stop`. Dès la fin du décodage du temps haut, la donnée est stabilisée et le drapeau `data_valid` est positionné à '1'. Il est réinitialisé lorsque `start_stop` repasse à '0'.
2. **Mode continu** (`continu = '1'`) : L'acquisition s'exécute automatiquement et rafraîchit périodiquement le registre d'angle chaque seconde.

2 Conception VHDL du Module Compas (`compass_choix_carte`)

Le bloc autonome de mesure a été développé dans l'architecture `compass_choix_carte.vhd`. Il comprend trois sous-fonctions principales synchronisées sur l'horloge système 50 MHz (`clk_50M`) :

2.1 Génération de la base de temps et de l'échantillonnage à 100 kHz

Pour mesurer directement le temps haut par pas de 10 μs à 100 μs, un processus de division génère une impulsion d'échantillonnage à 100 kHz :

$$N_{\text{echant}} = \frac{50 \text{ MHz}}{100 \text{ kHz}} = 500 \text{ cycles} \tag{2}$$

Un compteur périodique (`timer_3`) compte de 0 à 5000 (ou 500) pour générer l'horloge logique d'échantillonnage `echant_signal`. Parallèlement, un diviseur de fréquence (`timer_5`) compte 50×10^6 périodes de l'horloge système pour lever l'indicateur d'une seconde (`temp_1s <= '1'`) qui cadence le rafraîchissement continu.

2.2 Machine à états finis (FSM)

La gestion opérationnelle s'appuie sur une machine de Moore à 5 états :
— `idle` : État de repos après initialisation (`raz_n = '0'`).
— `continue` : Mode continu activé par `switch = '1'` (ou registre de contrôle).
— `attente_cont` : Temporisation après validation d'une mesure en attente de l'échéance de la seconde (`temp_1s`).
— `attente_mono` : Attente de la commande bouton `btn = '0'` en mode manuel.

— `mono` : Exécution d'une mesure monocoup unique jusqu'à détection de la fin du palier haut du PWM (`echantillon = '1'`).

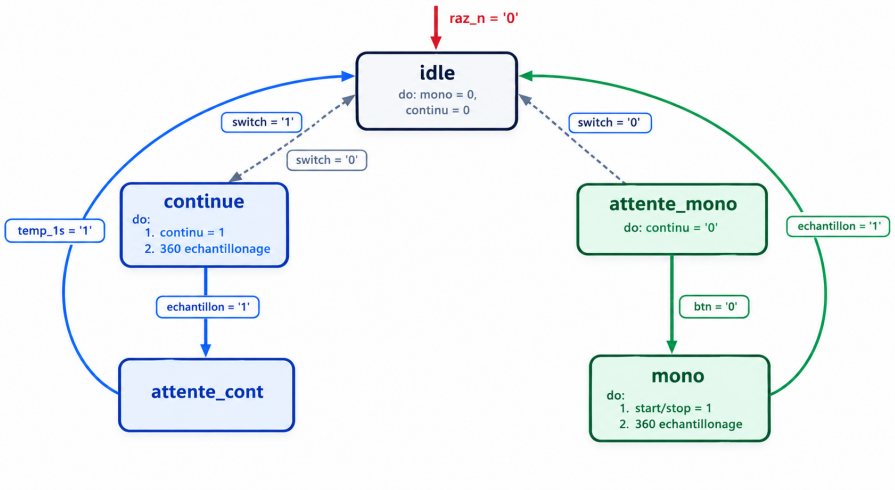


FIGURE 1 – Machine à états finis (FSM)

2.3 Décodage de la largeur d'impulsion

Le processus séquentiel détecte le front montant du signal PWM (`in_pwm`), arme le comptage, puis incrémente le registre interne `signal_registre` au rythme des fronts du signal d'échantillonnage tant que la ligne est au niveau logique haut. Au front descendant, le comptage cesse, la donnée est transférée dans le registre de sortie et le bit `data_valid` est asservi à '1'.

3 Simulation Temporelle sous ModelSim

Le banc de test `tb_compass_choix` a permis de valider la séquence d'initialisation, le fonctionnement monocoup déclenché par bouton, et le bouclage périodique en mode continu.

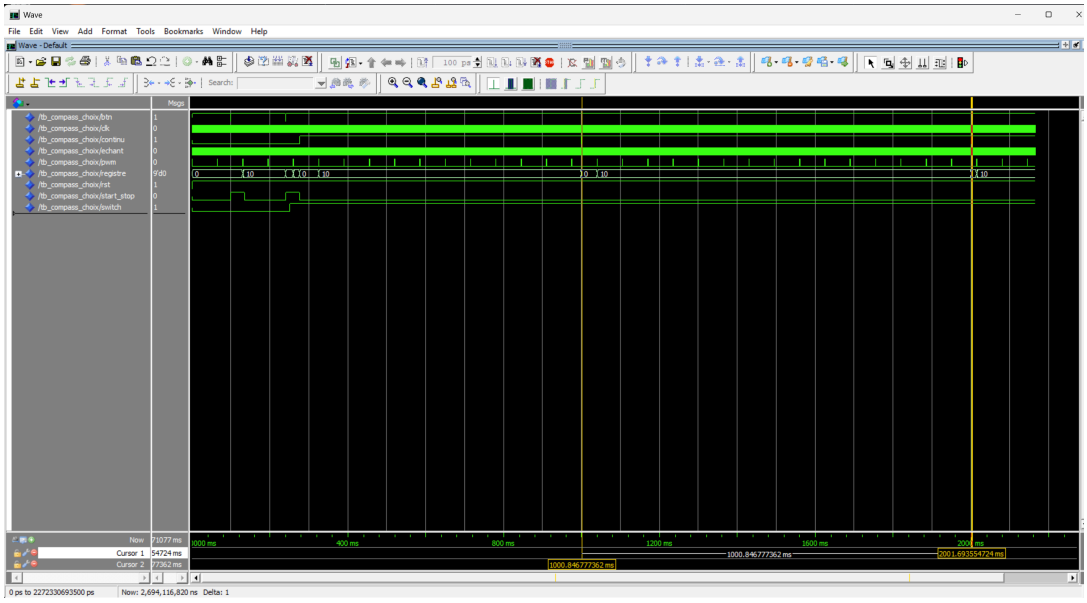


FIGURE 2 – Chronogramme de simulation ModelSim du module de cap (`tb_compass_choix`).

3.1 Analyse des résultats de simulation

L'observation du chronogramme (Figure 2) met en évidence les points suivants :

1. **Mesure du pas temporel de 1 seconde :** Les deux curseurs de mesure positionnés sur les fronts d'activation du mode continu indiquent respectivement $T_1 = 1000{,}846$ ms et $T_2 = 2001{,}693$ ms. Le delta obtenu :
$$\Delta T = 1000{,}8467 \text{ ms} \approx 1{,}00 \text{ seconde} \tag{3}$$
confirme l'exactitude de la base de temps cadencée par le compteur 50 MHz.
2. **Acquisition de la valeur d'angle :** Lors de l'injection d'une impulsion de test de référence, le registre de sortie se stabilise sur la valeur décimale `9'd10`.
3. **Comportement des signaux de contrôle :** Le signal `start_stop` s'active convenablement lors de la sollicitation monocoup, et `continu` passe à '1' lors de la sélection continue, conformément aux spécifications fonctionnelles.

4 Intégration SOPC & Interfaces Avalon-MM

L'objectif de cette étape est de relier notre composant matériel au processeur embarqué Nios II via le bus standard d'interconnexion Altera Avalon Memory-Mapped (Avalon-MM).

4.1 Composant esclave Avalon : `avalon_aquisition_cap`

Le module matériel a été encapsulé dans une interface esclave Avalon-MM. Le mapping des registres internes est le suivant :

Adresse relative	Accès	Registre	Description des bits
0x00	R/W	config	bit 0 : raz_n (Reset actif à 0) bit 1 : continu (1 = mode continu) bit 2 : start_stop (1 = déclenchement monocoup)
0x04	R	compas	bits 8..0 : data_compas (Cap réel de 0 à 359°) bit 9 : data_valid (1 = mesure disponible)

Table 1 – Affectation des registres Avalon de l'IP acquisition cap.

Le processus de lecture en VHDL décode l'adresse de mot :

```
process_Read: PROCESS(address, start_stop, continu, raz_n, data_compas_reg, sig_data_valid)
BEGIN
    case address is
        when "00" =>
            readdata <= x"0000000" & "0" & start_stop & continu & raz_n;
        when "01" =>
            readdata <= x"00000" & "00" & sig_data_valid & data_compas_reg;
        when others =>
            readdata <= (others => '0');
    end case;
END PROCESS process_Read;
```

4.2 Module d'émulation : `avalon_pwm`

Afin de tester le système en boucle fermée sans boussole physique connectée, un générateur PWM paramétrable sur le bus Avalon (`avalon_pwm.vhd`) a été intégré :

— `Adresse 0 (0x00)` : Registre `freq` fixant la période de récurrence.
— `Adresse 1 (0x04)` : Registre `duty` fixant la largeur haute de l'impulsion.
— `Adresse 2 (0x08)` : Registre de commande `control` (bit 0 = enable, bit 1 = division active).

4.3 Architecture globale sous Qsys / Platform Designer

La figure 3 illustre le schéma d'interconnexion élaboré sous Intel Platform Designer (Qsys).

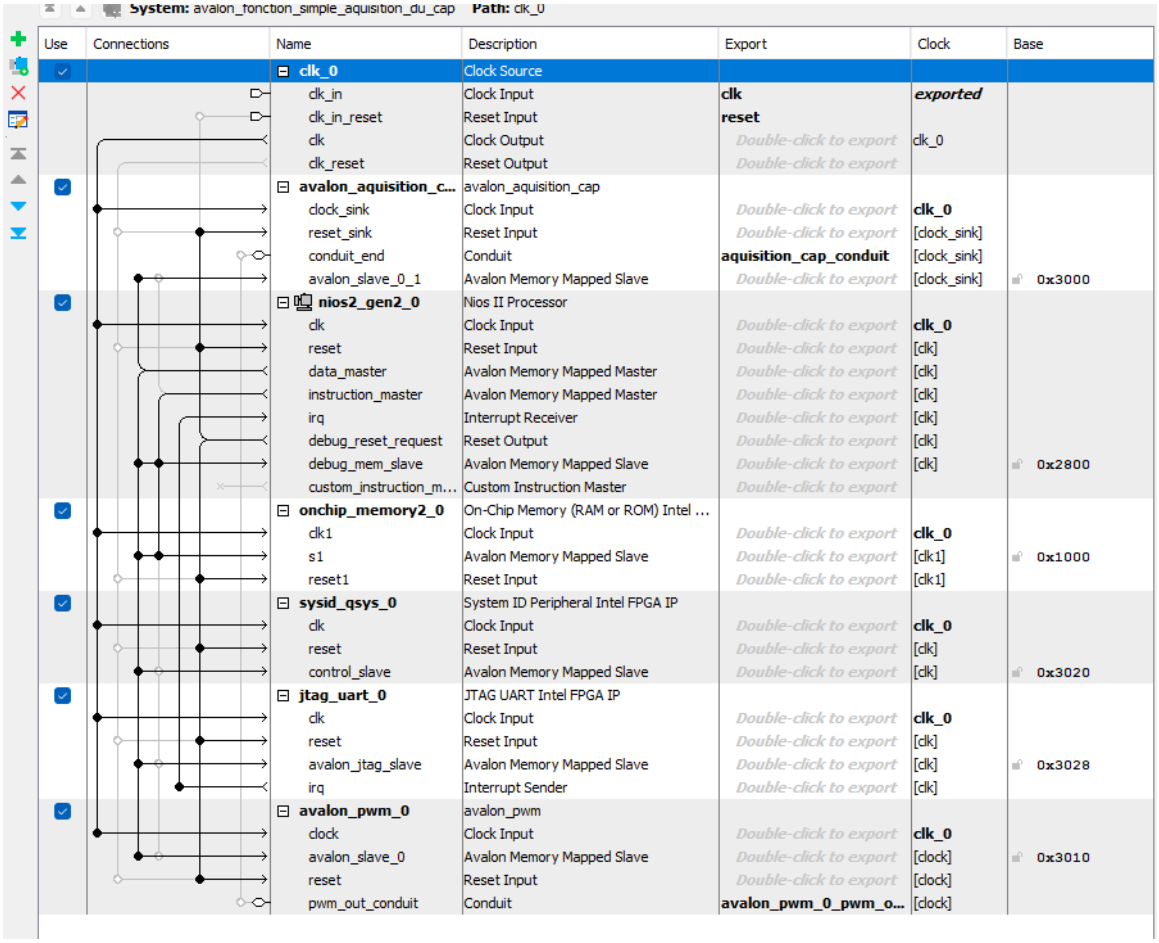


FIGURE 3 – Architecture du système SOPC sous Qsys / Platform Designer.

Le système comporte :
- Le processeur **nios2_gen2_0** (Master d'instructions et de données).
- La mémoire interne **onchip_memory2_0** mappée à l'adresse de base `0x1000`.
- L'IP d'identification système **sysid_qsys_0** à l'adresse `0x3020`.
- L'interface de communication console **jtag_uart_0** à l'adresse `0x3028` avec gestion des interruptions (`irq = 0`).
- Le composant personnalisé **avalon_aquisition_cap** connecté à l'adresse `0x3000`.
- Le générateur de test **avalon_pwm_0** connecté à l'adresse `0x3010`.

5 Schéma Bloc Quartus et Affectation des Broches

5.1 Schéma Bloc Top-Level (BDF)

L'ensemble du circuit SOPC généré est instancié dans le schéma bloc top-level Quartus (`avalon_fonction_simple_aquisition_du_cap.bdf`), relié aux ports d'entrées/sorties de la carte DE0-Nano.

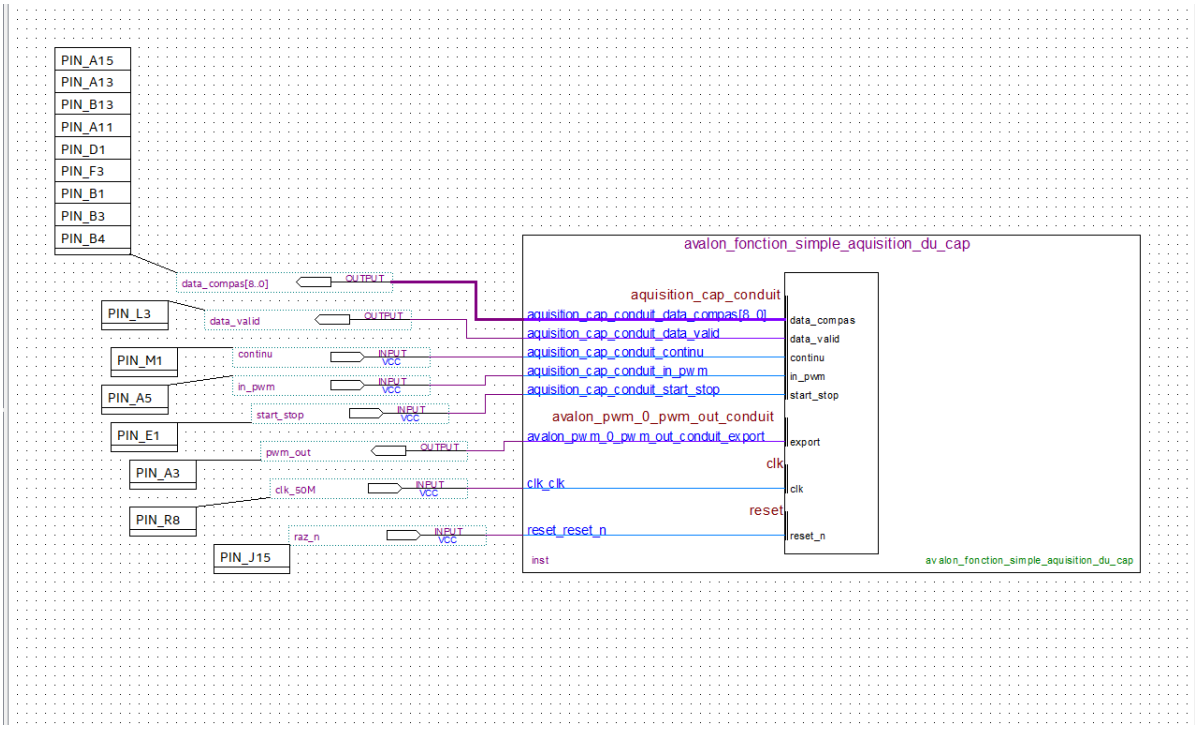


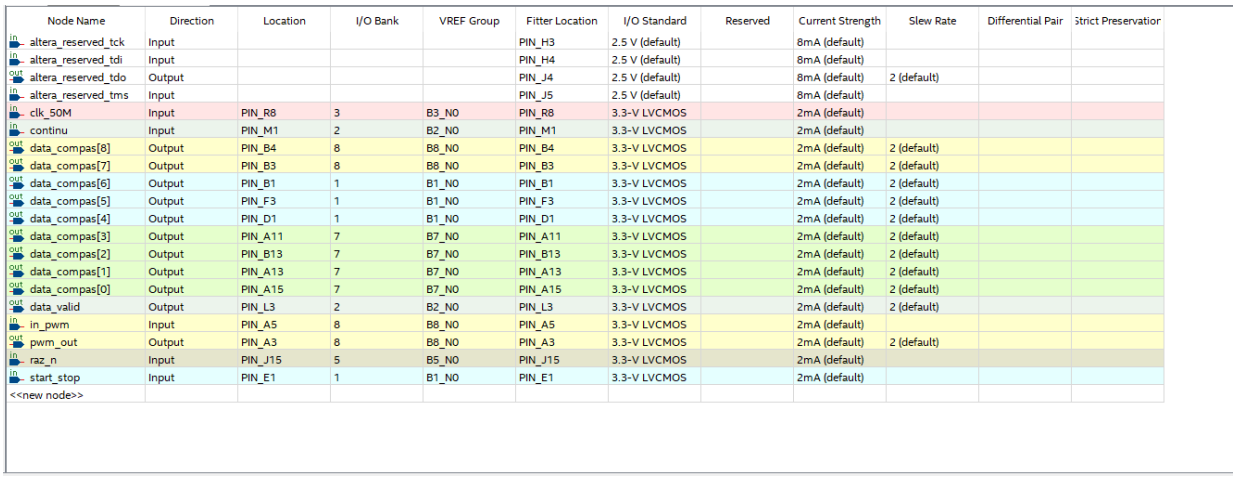
FIGURE 4 – Schéma bloc Quartus reliant l'entité SOPC aux broches physiques.

5.2 Affectation des broches (Pin Planner)

Le tableau suivant récapitule l'assignation physique réalisée sous Quartus Prime pour la carte Altera DE0-Nano (Cyclone IV EP4CE22F17C6N) :

Nom du port	Direction	Broche FPGA	Standard I/O	Rôle matériel
`clk_50M`	Entrée	`PIN_R8`	3.3-V LVCMOS	Horloge maître 50 MHz de la carte
`raz_n`	Entrée	`PIN_J15`	3.3-V LVCMOS	Bouton poussoir de reset général
`continu`	Entrée	`PIN_M1`	3.3-V LVCMOS	Switch de sélection mode continu
`start_stop`	Entrée	`PIN_E1`	3.3-V LVCMOS	Bouton d'acquisition monocoup
`in_pwm`	Entrée	`PIN_A5`	3.3-V LVCMOS	Entrée PWM de la boussole
`pwm_out`	Sortie	`PIN_A3`	3.3-V LVCMOS	Sortie PWM générée pour rebouclage
`data_valid`	Sortie	`PIN_L3`	3.3-V LVCMOS	Voyant LED de validation de mesure
`data_compas[8]`	Sortie	`PIN_B4`	3.3-V LVCMOS	Bit de poids fort (MSB) angle
`data_compas[7..0]`	Sorties	`PIN_B3..A15`	3.3-V LVCMOS	Bits d'angle affichés sur LED

TABLE 2 – Synthèse du plan de câblage sur la carte DE0-Nano.



Node Name	Direction	Location	I/O Bank	VREF Group	Fitter Location	I/O Standard	Reserved	Current Strength	Slew Rate	Differential Pair	Strict Preservation
altera_reserved_tck	Input				PIN_H3	2.5 V (default)		8mA (default)			
altera_reserved_tdi	Input				PIN_H4	2.5 V (default)		8mA (default)			
altera_reserved_tdo	Output				PIN_J4	2.5 V (default)		8mA (default)	2 (default)		
altera_reserved_tms	Input				PIN_J5	2.5 V (default)		8mA (default)			
clk_50M	Input	PIN_R8	3	B3_N0	PIN_R8	3.3-V LVCMOS		2mA (default)			
continu	Input	PIN_M1	2	B2_N0	PIN_M1	3.3-V LVCMOS		2mA (default)			
data_compas[8]	Output	PIN_B4	8	B8_N0	PIN_B4	3.3-V LVCMOS		2mA (default)	2 (default)		
data_compas[7]	Output	PIN_B3	8	B8_N0	PIN_B3	3.3-V LVCMOS		2mA (default)	2 (default)		
data_compas[6]	Output	PIN_B1	1	B1_N0	PIN_B1	3.3-V LVCMOS		2mA (default)	2 (default)		
data_compas[5]	Output	PIN_F3	1	B1_N0	PIN_F3	3.3-V LVCMOS		2mA (default)	2 (default)		
data_compas[4]	Output	PIN_D1	1	B1_N0	PIN_D1	3.3-V LVCMOS		2mA (default)	2 (default)		
data_compas[3]	Output	PIN_A11	7	B7_N0	PIN_A11	3.3-V LVCMOS		2mA (default)	2 (default)		
data_compas[2]	Output	PIN_B13	7	B7_N0	PIN_B13	3.3-V LVCMOS		2mA (default)	2 (default)		
data_compas[1]	Output	PIN_A13	7	B7_N0	PIN_A13	3.3-V LVCMOS		2mA (default)	2 (default)		
data_compas[0]	Output	PIN_A15	7	B7_N0	PIN_A15	3.3-V LVCMOS		2mA (default)	2 (default)		
data_valid	Output	PIN_L3	2	B2_N0	PIN_L3	3.3-V LVCMOS		2mA (default)	2 (default)		
in_pwm	Input	PIN_A5	8	B8_N0	PIN_A5	3.3-V LVCMOS		2mA (default)			
pwm_out	Output	PIN_A3	8	B8_N0	PIN_A3	3.3-V LVCMOS		2mA (default)	2 (default)		
raz_n	Input	PIN_J15	5	B5_N0	PIN_J15	3.3-V LVCMOS		2mA (default)			
start_stop	Input	PIN_E1	1	B1_N0	PIN_E1	3.3-V LVCMOS		2mA (default)			
<<new node>>											

FIGURE 5 – Vue de la configuration dans le Pin Planner de Quartus Prime.

6 Développement Logiciel Nios II et Validation

6.1 Programme applicatif C (`hello_world_small.c`)

Le logiciel embarqué sur le processeur Nios II initialise le périphérique de génération de signal PWM puis interroge cycliquement l'interface Avalon du compas :

```c
#include <stdio.h>
#include <unistd.h>
#include "io.h"
#include "system.h"
#include "alt_types.h"
#include "sys/alt_stdio.h"

#define CAP_DATA_REG    (1 * 4)
#define PWM_FREQ_REG    (0 * 4)
#define PWM_DUTY_REG    (1 * 4)
#define PWM_CTRL_REG    (2 * 4)

int main() {
    alt_putstr("Start\n");

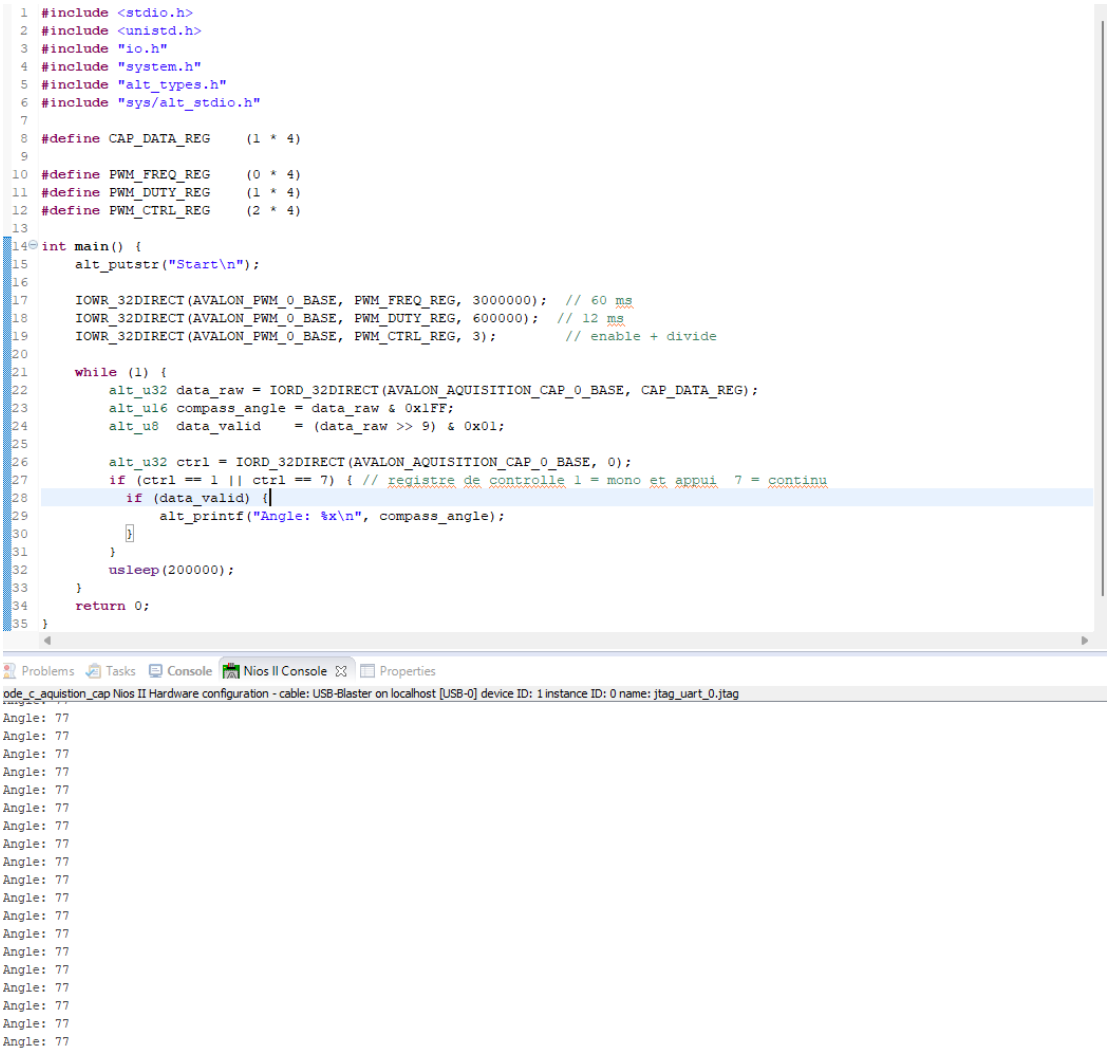
    // Configuration du generateur PWM de test
    // Horloge = 50 MHz : 3 000 000 coups = 60 ms de periode
    IOWR_32DIRECT(AVALON_PWM_0_BASE, PWM_FREQ_REG, 3000000);
    // 600 000 coups = 12 ms de largeur haute
    IOWR_32DIRECT(AVALON_PWM_0_BASE, PWM_DUTY_REG, 600000);
    // Activation de la PWM et du diviseur
    IOWR_32DIRECT(AVALON_PWM_0_BASE, PWM_CTRL_REG, 3);

    while (1) {
        alt_u32 data_raw = IORD_32DIRECT(AVALON_AQUISITION_CAP_0_BASE, CAP_DATA_REG);
        alt_u16 compass_angle = data_raw & 0x1FF;
        alt_u8  data_valid    = (data_raw >> 9) & 0x01;

        alt_u32 ctrl = IORD_32DIRECT(AVALON_AQUISITION_CAP_0_BASE, 0);
        if (ctrl == 1 || ctrl == 7) {
            if (data_valid) {
                alt_printf("Angle: %x\n", compass_angle);
            }
        }
        usleep(200000);
    }
    return 0;
}
```

6.2 Résultats d'exécution sur la Console Nios II

En reliant physiquement la sortie `pwm_out` (`PIN_A3`) à l'entrée `in_pwm` (`PIN_A5`) sur la carte DE0-Nano, le système a été validé en temps réel.



```c
#include <stdio.h>
#include <unistd.h>
#include "io.h"
#include "system.h"
#include "alt_types.h"
#include "sys/alt_stdio.h"

#define CAP_DATA_REG    (1 * 4)

#define PWM_FREQ_REG    (0 * 4)
#define PWM_DUTY_REG    (1 * 4)
#define PWM_CTRL_REG    (2 * 4)

int main() {
    alt_putstr("Start\n");

    IOWR_32DIRECT(AVALON_PWM_0_BASE, PWM_FREQ_REG, 3000000);  // 60 ms
    IOWR_32DIRECT(AVALON_PWM_0_BASE, PWM_DUTY_REG, 600000);  // 12 ms
    IOWR_32DIRECT(AVALON_PWM_0_BASE, PWM_CTRL_REG, 3);        // enable + divide

    while (1) {
        alt_u32 data_raw = IORD_32DIRECT(AVALON_AQUISITION_CAP_0_BASE, CAP_DATA_REG);
        alt_u16 compass_angle = data_raw & 0x1FF;
        alt_u8  data_valid    = (data_raw >> 9) & 0x01;

        alt_u32 ctrl = IORD_32DIRECT(AVALON_AQUISITION_CAP_0_BASE, 0);
        if (ctrl == 1 || ctrl == 7) { // registre de controlle 1 = mono et appui  7 = continu
          if (data_valid) {
              alt_printf("Angle: %x\n", compass_angle);
          }
        }
        usleep(200000);
    }
    return 0;
}
```

Problems Tasks Console Nios II Console Properties

ode_c_aquistion_cap Nios II Hardware configuration - cable: USB-Blaster on localhost [USB-0] device ID: 1 instance ID: 0 name: jtag_uart_0.jtag

```
Angle: 77
Angle: 77
Angle: 77
Angle: 77
Angle: 77
Angle: 77
Angle: 77
Angle: 77
Angle: 77
Angle: 77
Angle: 77
Angle: 77
Angle: 77
Angle: 77
Angle: 77
Angle: 77
Angle: 77
Angle: 77
Angle: 77
Angle: 77
```

FIGURE 6 – Traces d'exécution affichées dans la console Nios II.

6.3 Interprétation physique de la mesure

La console Nios II affiche en continu la valeur hexadécimale `Angle: 77` :

$$(77)_{16} = 7 \times 16 + 7 = 119_{10} \tag{4}$$

Or, la configuration du générateur de signal injectait un temps haut défini par :

$$t_{\text{haut}} = \frac{600\,000}{50 \times 10^6 \text{ Hz}} = 12 \text{ ms} \tag{5}$$

Étant donné la fréquence d'échantillonnage de 100 kHz (période d'échantillonnage de 100 μs), le nombre de coups d'échantillon comptabilisés sur une durée de 12 ms est égal à :

$$N = \frac{12 \text{ ms}}{100\,\mu\text{s}} = 120 \text{ incréments} \tag{6}$$

La valeur obtenue expérimentalement (119_{10}) correspond exactement au modèle théorique à un quantum d'échantillonnage près (120 ± 1), démontrant la précision et la synchronisation du module matériel.

7 Conclusion

Ce bureau d'études a permis de franchir les étapes complètes de conception sur FPGA :

— La description VHDL du décodage de cap CMPS03/10 selon les deux modes imposés (monocoup et continu).
— La simulation fonctionnelle sous ModelSim validant le cadencement temporel de 1 seconde et le traitement d'impulsion.
— L'encapsulation sous forme d'IP compatible avec le bus Avalon Memory-Mapped.
— L'intégration SOPC complète sous Intel Qsys avec processeur Nios II, mémoire vive et module générateur PWM de rebouclage.
— L'écriture du driver C de bas niveau et la validation matérielle directe sur la carte DE0-Nano.